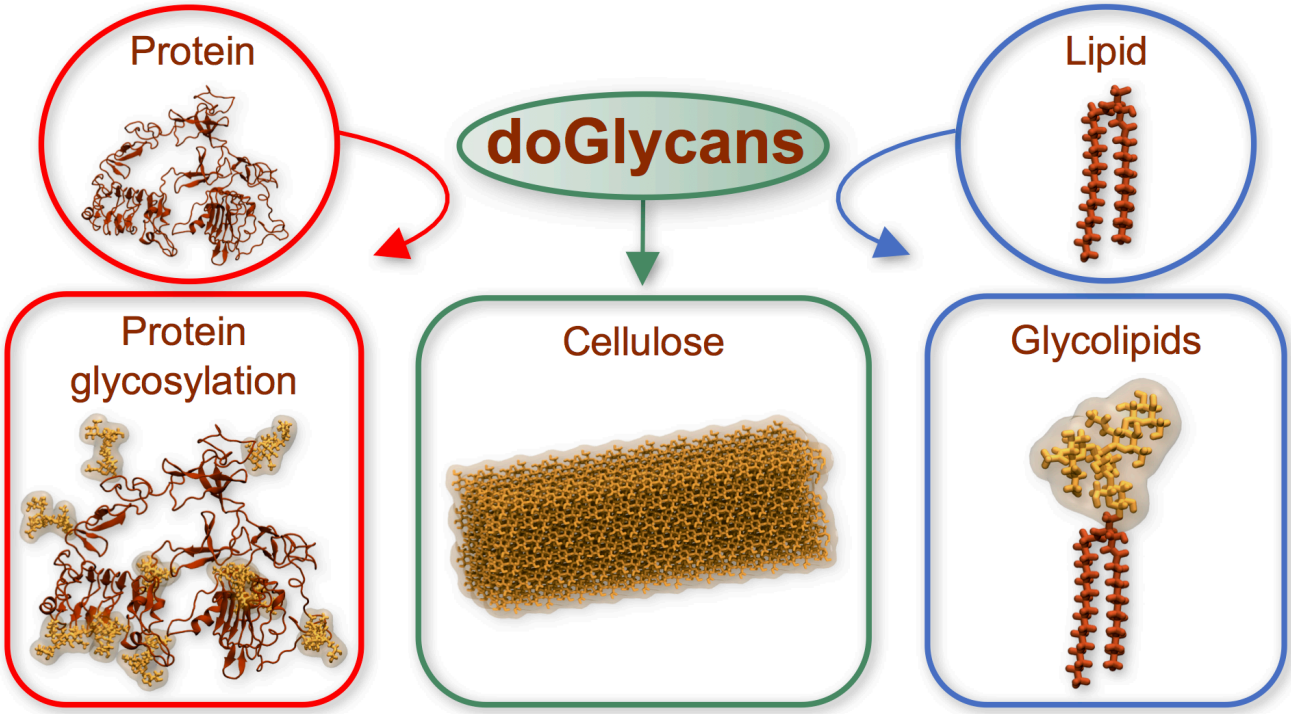
doGlycans

Tools for Preparing Carbohydrate Structures for Atomistic Simulations of Glycoproteins, Glycolipids, and Carbohydrate Polymers for GROMACS



USER MANUAL

doGlycans

USER MANUAL

doGlycans – Tools for Preparing Carbohydrate Structures for Atomistic Simulations of Glycoproteins, Glycolipids, and Carbohydrate Polymers for GROMACS

Reinis Danne[a,‡], Chetan Poojari[a,b,‡], Hector Martinez-Seara[a,c], Sami Rissanen[a], Fabio Lolicato[a,b], Tomasz Róg[a,b,*], Ilpo Vattulainen[a,b,d,*]

[a] Laboratory of Physics, Tampere University of Technology, P. O. Box 692, FI-33101 Tampere, Finland
[b] Department of Physics, University of Helsinki, P. O. Box 64, FI-00014, Finland
[c] Institute of Organic Chemistry and Biochemistry, Academy of Sciences of the Czech Republic, Flemingovo nám. 2, 16610 Prague 6, Czech Republic
[d] MEMPHYS – Center for Biomembrane Physics, University of Southern Denmark, Odense, Denmark

[*]**To whom correspondence should be forwarded: Ilpo.Vattulainen@helsinki.fi, tomasz.rog@gmail.com**
[‡]**These authors contributed equally to the work**

The present ***doGlycans*** user manual will guide users of this software package to prepare structures and topology files needed for simulations of **glycoproteins, glycolipids, and carbohydrate polymers** for **GROMACS** molecular dynamics simulation package. This software allows using the **Glycan06** force field for sugars in combination with **OPLS-AA, AMBER, and CHARMM** force fields.

Disclaimer

The programs and scripts provided in the *doGlycans* package are 'as-is', without any express or implied warranty. In no event will the authors be held liable for any damages arising from the use of this software. The *doGlycans* tool and the manual will be updated if any additional features are added to the tool. The users are requested to always use the latest version of the tool. Any comments or issues in the usage of the tool can be posted by creating an issue in the *doGlycans* public repository:

https://bitbucket.org/biophys-uh/doglycans/

Citing *doGlycans*

Reinis Danne, Chetan Poojari, Hector Martinez-Seara, Sami Rissanen, Fabio Lolicato, Tomasz Róg, Ilpo Vattulainen. doGlycans – Tools for Preparing Carbohydrate Structures for Atomistic Simulations of Glycoproteins, Glycolipids, and Carbohydrate Polymers for GROMACS. J. Chem. Inf. Model. (2017) [Please update the reference not known in full at the time of writing this manual].

doGlycans is Free Software

The *doGlycans* tool set is a free software package and can be redistributed and/or modified under the terms of the GNU General Public License as published by the Free Software Foundation, version 3 of the License. For more details, see

https://www.gnu.org/licenses/gpl-3.0.en.html

The *doGlycans* source code can be downloaded from

https://bitbucket.org/biophys-uh/doglycans.git

doGlycans is `doglycans`

In this manual, the `doglycans` as a command is given in lowercase. Therefore, on a command level, "`doGlycans`" will be and should be given as "`doglycans`".

For more information on the research activities of the team standing behind the tool, please visit the webpage

https://www.helsinki.fi/en/researchgroups/biophysics

Table of Contents

1. INSTALLATION OF THE *DOGLYCANS* TOOL 5
2. SEQUENCE PREPARATION 7
3. CARBOHYDRATE POLYMERS 15
4. GLYCOSYLATION OF PROTEINS 17
4.1 Processing the Protein Data Bank file using the GROMACS tool *pdb2gmx* *17*
4.2 Preparing the sequence file *18*
4.2.1 Preparing the sequence file for glycosylation of EGFR *18*
4.2.2 Preparing the sequence file for N- and O- linked glycosylation of CD59 ..... *20*
4.3 Adding sugars to the structure file using the doGlycans tool *21*
4.4 Preparing the topology file for MD Simulations: the OPLS-AA force field *24*
4.5 Preparing the topology file for MD Simulations: the AMBER force field *25*
5. PROCEDURES TO PREPARE GLYCANS FOR GLYCOLIPIDS 27

1. Installation of the *doGlycans* tool

The dependencies can be installed by using one of the following two methods.

1.1 Installation of dependences

1.1a Recommended installation using Anaconda Cloud:

Anaconda is a self-contained and customizable python distribution and its libraries can be downloaded from its web page:

https://www.continuum.io/downloads

Please download the latest python 3.X version of the package and install it.

```
sh Anaconda3-4.3.1-Linux-x86_64.sh
```

At the end of the installation process you will be offered a chance to prepend the Anaconda3 install location to your path. This is required to have access to the executables. This essentially appends these two lines to `~/.bashrc` or `~/.bash_profile` for Mac OS X systems

```
# added by Anaconda3 X.X.X installer
export PATH="/INSTALL_PATH/anaconda3/bin:$PATH"
```

Although this is fine for most of the users, some can experience certain problems. This is because your default python interpreter may be replaced by the one shipped to anaconda. This can be avoided by reverting the look up order:

```
# added by Anaconda3 X.X.X installer
export PATH="$PATH:/INSTALL_PATH/anaconda3/bin"
```

After this one can create a customized environment to run the *doGlycans* tools:

```
conda create --name doglycans "python>=3.6" "numpy=1.13.1" "ply>=3.10"
```

To enter the *doGlycans* environment we type:

```
source activate doglycans
```

We finish customizing the environment by installing openbabel:

```
conda install -c openbabel openbabel=2.4.1
```

To exit the *doGlycans* environment, either close the used terminal or type:

```
source deactivate doglycans
```

1.1b Manual installation:
For manual installation, the users are required to install the following dependencies to run the *doGlycans* tool. We expect users to have PYTHON 3 (>3.6) installed (the *doGlycans* tool was tested with PYTHON 3.6).

- `swig-3.0.12 or greater (for openbabel python bindings)`
- `openbabel 2.4.1 or greater`
- `numpy 1.13.1 or greater`
- `ply 3.10 or greater`

Please see the `REFERENCES.txt` file for their relative web pages.

1.2 Downloading, installing and running the *doGlycans* tool

Following the successful installation of the dependencies, the users are ready to use *doGlycans* tool to glycosylate their systems. You just need to download the latest version from:

`https://bitbucket.org/biophys-uh/doglycans.git`

Notice that the program is self-contained in the folder together with the examples. Do not alter the content of the folder or move it out of the folder, as it would no longer work.

It is recommended to add a variable pointing to the folder path to avoid writing the path several times per command. For this, please add the following to your .bashrc or .profile (this is assumed in the examples given below):

```
# doglycans path
DGPATH="/PATH/TO/doglycans/"
```

To run the *doGlycans,* execute the desired script within the newly created environment:

```
source activate doglycans
python $DGPATH/PROGRAM.py [options]
```

1.3 Additional dependencies for running the examples and test files:

As the *doGlycans* tool generates input files for simulations to be carried out in the GROMACS molecular dynamics (MD) simulation package, for taking advantage of its functionalities the users should have the GROMACS package installed.

GROMACS can be found in `http://www.gromacs.org/Downloads`.

All examples presented here have been prepared to be run in GROMACS 4.6.x. Newer versions require minor changes in the parameters files, i.e., mdp files.

2. Sequence preparation

One basic file in the *doGlycans* programs is the sequence file. This file contains the sequence of the sugar one wants to construct, and also the information of where it is attached when needed, e.g., in a protein. The naming of the sugars residues in the sequence file is based on the GLYCAM naming convention. In this convention, a three letter code is used to refer precisely to a given sugar and its linkage. This compact PDB friendly convention includes carbohydrate name, configuration, and linkage position. Details of the sugars can be found in the GLYCAM monosaccharides database (PREP file), which is used by the *doGlycans* code. In Table 1, we provide the list of carbohydrates and their one/two letter codes. Notice that the letters in upper case refer to the D-configuration of the monosaccharide, while the lower case is used for the L-monosaccharides. The one-letter codes are followed by either A or B to refer to the Alpha or Beta isomer in the anomeric carbon. Finally, the code is preceded by the A letter referring to its connectivity, see Table 2. The compact format used in the sequence file, which is admittedly complicated, aims to follow the standards set by the RSCB PDB advisory Committee for PDB files and it is consistent with the three-letter codes required in PDB as for amino acids.

Table 1: The single letter codes for monosaccharides used in GLYCAM and in the *doGlycans* tool.

Monosaccharides	**One letter code**
D-Arabinose	A
D-Lyxose	D
D-Ribose	R
D-Xylose	X
D-Allose	N
D-Altrose	E
D-Galactose	L
D-Glucose	G
D-Gulose	K
D-Idose	I
D-Mannose	M
D-Talose	T
D-Fructose	C
D-Psicose	P
D-Sorbose	B
D-Tagatose	J
D-Fucose	F
D-Quinovose	Q
D-Rhamnose	H
D-Galacturonic Acid	O
D-Glucuronic Acid	Z
D-Iduronic Acid	U
D-N-Acetylgalactosamine	V
D-N-Acetylglucosamine	Y
D-N-Acetylmannosamine	W
N-Acetyl-neuraminic Acid	S
KDN	KN
KDO	KO
N-Glycolyl-neuraminic Acid	SG

The single-letter code for all the D-enantiomeric forms are in capital letters as given in Table 1. For L-enantiomeric forms the single-letter codes are written in small letters. For example, L-Glucose

would be "g". The less common monosaccharides such as KDN, KDO, N-Glycolyl-neuraminic Acid can be assigned a two-letter code.

Table 2: The linkage positions and three-letter code used in GLYCAM and by *doGlycans* tool for glucopyranose in alpha-configuration (The alpha configuration is signal with the terminal A, opposed to the B configuration that will use a terminal B)

Linkage Positions	**Three-letter code**
Terminal[b]	0GA[b]
1-[c]	1GA[c]
2-	2GA
3-	3GA
4-	4GA
6-	6GA
2,3-	ZGA
2,4-	YGA
2,6-	XGA
3,4-	WGA
3,6-	VGA
4,6-	UGA
2,3,4-	TGA
2,3,6-	SGA
2,4,6-	RGA
3,4,6-	QGA
2,3,4,6-	PGA

[b]Here the zero prefix indicates chain non-reducing end termination, *i.e.,* the terminal sugar has no free oxygen atoms for bond formation.
[c]Used for the 1-1 linkage position.

For glucopyranoses in the beta-configuration, the three-letter code will end with the letter "B". For example, the β-D-glucose residue with a linkage position at 4- would be written as "4GB". In the case of the β-D-mannose residue with a 3,6- linkage, the three-letter code would be "VMB". Here, the letter "V" indicates 3,6- linkage, "M" indicates mannose sugar and "B" refers to beta configuration.

In the case of furanoses, the single-letter codes used to describe the linkage positions and monosaccharides remain the same, the only change here is that the alpha configuration is written as "D" and the beta configuration is written as "U". For example, α-D-glucofuranose with a linkage position at 4- is written as 4GD and β -D-glucofuranose with a linkage position at 4- is written as 4GU.

The single-letter codes for L-enantiomeric forms are written in small letters. For example, the α-L-glucose residue with a linkage position at 3- would be "3gA"and the β-L-mannose residue with a linkage position at 2- would be "2mB".

Table 3: The linkage positions and the three-letter code used in GLYCAM and in the *doGlycans* tool for special monosaccharide derivatives.

Monosaccharide	One-letter code	Linkage Positions	Three-letter code
α-D-Sialic Acid	S	Terminal (non-reducing)	0SA
		7-	7SA
		8-	8SA
		9-	9SA
α-D-N-Acetylglucosamine	Y	Terminal (non-reducing)	0YA
		3-	3YA
		4-	4YA
		6-	6YA
		3,4-	WAY
		3,6-	VYA
		4,6-	UYA
		3,4,6-	RYA

The sequence file syntax consists basically of the sequence of monosaccharides starting from the reducing end preceded by its terminal group and terminating in the non-reducing end.

Example 1: The sequence for a monosaccharide β-D-glucose will be written as follows:

```
monosaccharide = ROH-(O1,C1)-0GB
```

Example 2: The sequence for a 1,4 linked β-D-glucose disaccharide will be written as follows:

```
disaccharide = ROH-(O1,C1)-4GB-(O4,C1)-0GB
```

Notice that in parenthesis one has to specify the atoms involved in the link. These atoms follow the usual naming convention for sugar moieties: O{1-6} or C{1-6} in most of the cases. For others please refer to the GLYCAM prep file included in the package.

Example 3: The sequence for a simple carbohydrate polymer consisting of a linear chain of 15 D-glucose (β-1,4 linked) units is written as below:

```
polymer=ROH-(O1,C1)-{4GB-(O4,C1)}14-0GB
```

Example 4: The same sequence as above where we explicitly set the dihedral angles of the glycosidic bonds

```
polymer=ROH-(O1,C1)-{4GB-(O4,C1,<label1>)}14-0GB
label1=O4,C1,[H4 C4 O4 C1 0.0 C4 O4 C1 H1 60.0]
```

The first line of the sequence file consists of information on the saccharide and its linkage (please refer to the GLYCAM naming convention for description of nomenclatures and linkages: http://glycam.org/docs/forcefield/glycam-naming-2/ and Table 1,2,3 of this manual). Here, ROH represents an addition of the terminal –OH group to the saccharide (D-glucose). The terminal –OH group is connected to the C1 atom of D-glucose which is given in parentheses as (O1,C1). Next,

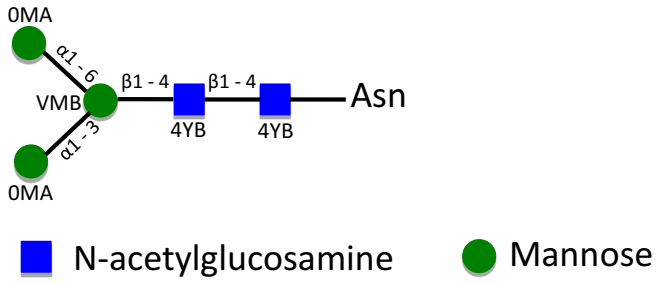
within curly brackets we enclose a single unit of β-D-glucose residue (4GB: number 4 indicates the open linkage position, G refers to one letter code to glucose, and B refers to beta configuration) specifying their linkage sites (O4,C1: linkage positions). An arbitrary label in the angle brackets, *i.e.,* <label1> can be used to specify dihedral connections for O4,C1. Then, the curly brackets are followed by an integer specifying the number of times the block (that is specified within the curly brackets) has to be repeated. The residue chain is terminated with "-0GB". Here, the zero prefix indicates the unavailability of oxygen atoms for further bond formation.

The single letter code in the angle brackets <label> is linked to the second line of the sequence file, where it contains information about the dihedral connections for the O4,C1 atoms as well as the dihedral angles with their values. Thus, the label used in the angle brackets <label1> to define dihedral connections must match the label "label1" used in the second line; if not, then the *doGlycans* code will not be able to assign specific dihedral connections and their values to the O4,C1 atoms.

There are no spaces on the first line of the sequence, whereas on the second line a single space is needed between atom names and angles for the dihedral specification within square brackets.

Example 5: Here we explain how to write more complex sequences with multiple branching, and how to assign multiple dihedral connections. Sequence file for attaching a simple Mannose type N-glycan ($GlcNAc_2Man_3$) to a protein is written as:

Mannose Type N-glycan ($GlcNAc_2Man_3$)

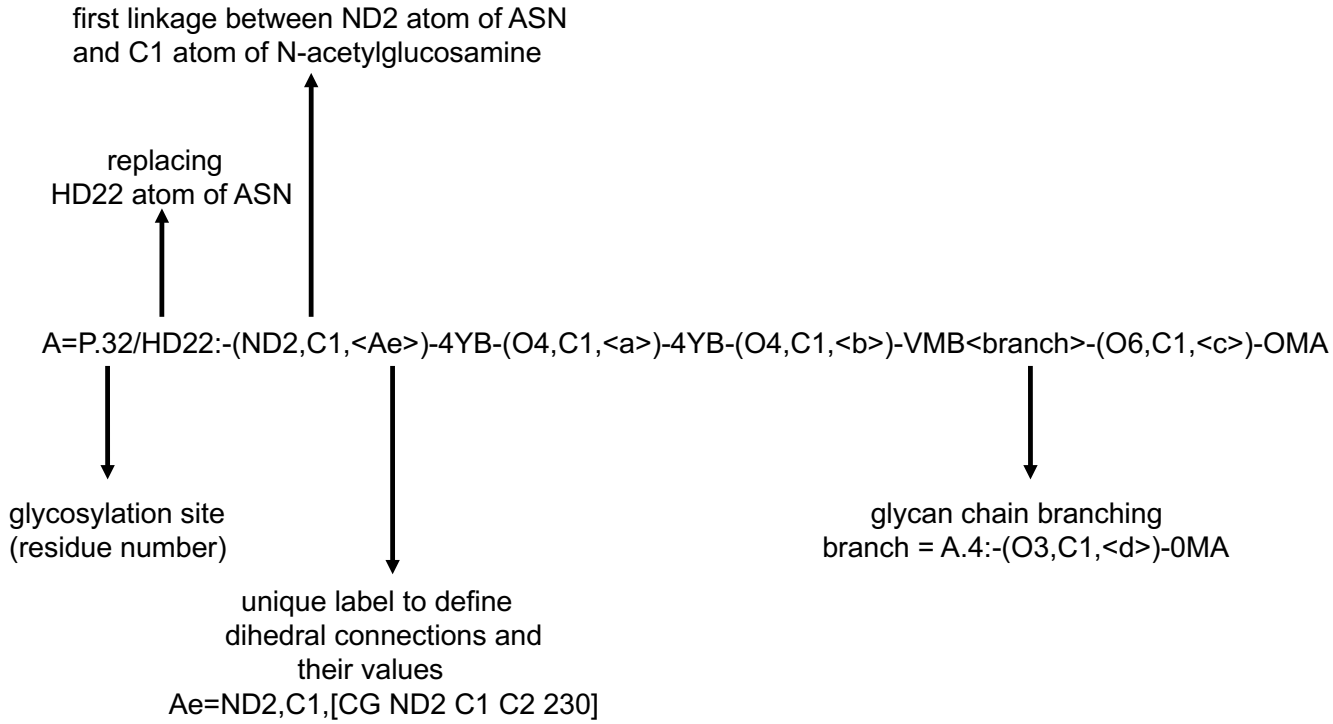



```
A=P.32/HD22:-(ND2,C1,<Ae>)-4YB-(O4,C1,<a>)-4YB-(O4,C1,<b>)-VMB<branch>-
(O6,C1,<c>)-0MA

branch = A.4:-(O3,C1,<d>)-0MA
a = O4,C1,[C3 C4 O4 C1   60.0 C4 O4 C1 C2 120.0]
b = O4,C1,[C3 C4 O4 C1  100.0 C4 O4 C1 C2  60.0]
c = O6,C1,[C4 C5 C6 O6   120.0 C5 C6 O6 C1 150.0 C6 O6 C1 C2 180.0]
d = O3,C1,[C2 C3 O3 C1  120.0 C3 O3 C1 C2 -60.0]
Ae=ND2,C1,[CG ND2 C1 C2 230]
```

In this example sequence, we show the attachment of Mannose type N-glycan ($GlcNAc_2Man_3$) to the protein by replacing the hydrogen atom HD22 of an Asparagine residue. The beginning of the first line defines the glycosylation site on the protein and the atom to be replaced (32/HD22: the number 32 indicates the glycosylation site and HD22 indicates the atom to be replaced (in the

asparagine residue)). This is followed by the first linkage defined within parentheses (ND2,C1,<Ae>) between the ND2 atom of Asparagine and the C1 atom of N-acetylglucosamine (the first sugar attached), and their dihedral linkage is defined by a single-letter code in the angle brackets <Ae>. After defining the initial attachment, the monosaccharides are added by their single-letter code, configuration, and linkage position. In the above example, the next monosaccharide added to the sequence is N-acetylglucosamine in beta configuration with an open linkage position at atom C4 (4YB). Their linkage position and a single-letter code defining their dihedral values are given within parentheses (`O4,C1,<a>`). We continue to build the linear chain of monosaccharides in a similar manner. Whenever there is a branching in the chain, we define it within the three-letter code used to define the monosaccharide. In the above example, the sugar branches at the mannose monosaccharide are given by its three-letter code VMB (VMB: M refers to mannose monosaccharide, B refers to beta configuration, and V refers to a single-letter code for branching at linkage positions 3,6-). The branching at the position 6-, their atom connections, and dihedrals are defined on the same line (O6,C1,<c>), whereas for branching at the position 3- we define it within angle brackets as <branch>, and on the following line using the same code "branch" we define the position 3- atom connections and their dihedrals (O3,C1,<d>). Each of the branches is terminated with "-0MA"; here, the zero prefix indicates the unavailability of oxygen atoms for bond formation and M refers to mannose, and A stands for alpha configuration. After each linkage, a unique label within the angle brackets specifies the connections between the two atoms as well as the dihedral angles with their values. The information about the dihedral connections and their values are given on separate lines. The schematic description of the sequence is given below:



If the same mannose type N-glycan ($GlcNAc_2Man_3$) has to be added at multiple glycosylation sites, it is not required to define the dihedral connections and their values for each site separately. The *doGlycans* tool will read the unique label within the angle brackets and assign the dihedral values accordingly at each glycosylation site. The sequence for a mannose type N-glycan at positions 32, 151 and 172 can be written as:

```
A=P.32/HD22:-(ND2,C1,<Ae>)-4YB-(O4,C1,<a>)-4YB-(O4,C1,<b>)-VMB<branch>-
(O6,C1,<c>)-0MA

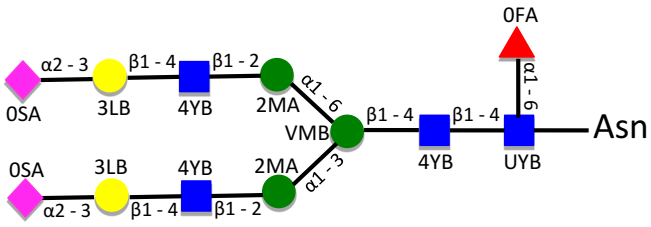
branch = A.4:-(O3,C1,<d>)-0MA
a = O4,C1,[C3 C4 O4 C1   60.0 C4 O4 C1 C2 120.0]
b = O4,C1,[C3 C4 O4 C1  100.0 C4 O4 C1 C2  60.0]
c = O6,C1,[C4 C5 C6 O6   120.0 C5 C6 O6 C1 150.0 C6 O6 C1 C2 180.0]
d = O3,C1,[C2 C3 O3 C1  120.0 C3 O3 C1 C2 -60.0]
Ae=ND2,C1,[CG ND2 C1 C2 230]

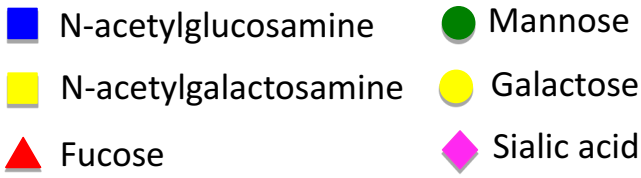
B=P.151/HD22:-(ND2,C1,<Ae>)-4YB-(O4,C1,<a>)-4YB-(O4,C1,<b>)-VMB<branch>-
(O6,C1,<c>)-0MA

C=P.172/HD22:-(ND2,C1,<Ae>)-4YB-(O4,C1,<a>)-4YB-(O4,C1,<b>)-VMB<branch>-
(O6,C1,<c>)-0MA
```

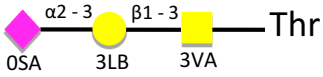
Example 6: However, if the glycan added is complex and differs for each site, then a separate dihedral value can be set for each glycosylation site with unique labels. Here we show how to write such a sequence with a complex type N-glycan ($GlcNAc_4Man_3Gal_2FucNeuNAc_2$) linked at residue 18 and the Mucin type O-glycan (GalNAcGalNeuNAc) linked at residue 52:

a) Complex Type N-glycan ($GlcNAc_4Man_3Gal_2FucNeuNAc_2$):





b) Mucin Type O-glycan (GalNAcGalNeuNAc):




```
E=P.18/HD22:-(ND2,C1,<Eh>)-UYB<EbranchFuc>-(O4,C1,<Ea>)-4YB-(O4,C1,<Eb>)-
VMB<EbranchMan>-(O6,C1,<Ec>)-2MA-(O2,C1,<Ed>)-4YB-(O4,C1,<Ei>)-3LB-
(O3,C2,<Ej>)-0SA

EbranchFuc = E.4:-(O6,C1,<Ee>)-0FA
EbranchMan = E.4:-(O3,C1,<Ef>)-2MA-(O2,C1,<Eg>)-4YB-(O4,C1,<Ek>)-3LB-
(O3,C2,<El>)-0SA
Ea = O4,C1,[C3 C4 O4 C1   119.364 C4 O4 C1 C2 159.284]
```

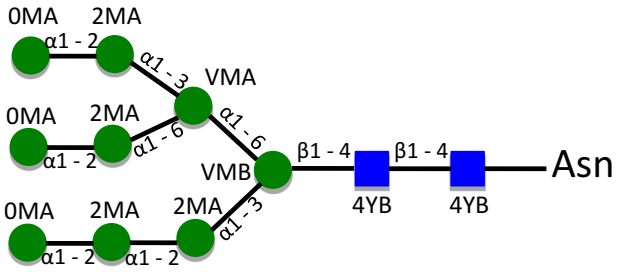
```
Eb = O4,C1,[C3 C4 O4 C1  109.335 C4 O4 C1 C2  156.828]
Ec = O6,C1,[C4 C5 C6 O6   63.058 C5 C6 O6 C1 142.635 C6 O6 C1 C2 -
177.591]
Ed = O2,C1,[C3 C2 O2 C1  119.036 C2 O2 C1 C2  148.221]
Ee = O6,C1,[C4 C5 C6 O6   68.073 C5 C6 O6 C1 -143.104 C6 O6 C1 C2
142.538]
Ef = O3,C1,[C2 C3 O3 C1  -87.136 C3 O3 C1 C2 -167.302]
Eg = O2,C1,[C3 C2 O2 C1  176.592 C2 O2 C1 C2  -173.747]
Ei = O4,C1,[C3 C4 O4 C1  109.335 C4 O4 C1 C2  156.828]
Ej = O3,C2,[C2 C3 O3 C2  -87.136 C3 O3 C2 C3 -167.302]
Ek = O4,C1,[C3 C4 O4 C1  109.335 C4 O4 C1 C2  156.828]
El = O3,C2,[C2 C3 O3 C2  -87.136 C3 O3 C2 C3 -167.302]
Eh=ND2,C1,[CB CG ND2 C1 78.223 CG ND2 C1 C2 61.491]

RD=P.52/HG1:-(OG1,C1,<RDe>)-3VA-(O3,C1,<Rb3>)-3LB-(O3,C2,<Rbd>)-0SA

Rb3 = O3,C1,[C2 C3 O3 C1  145.0 C3 O3 C1 C2 100.0]
Rbd = O3,C2,[C2 C3 O3 C2  -87.136 C3 O3 C2 C3 -167.302]
RDe=OG1,C1,[CB OG1 C1 C2 60 OG1 C1 C2 C3 115 CG2 CB OG1 C1 30]
```

Example 7: Preparing the sequence file for complex glycans

A) High-mannose Type N-glycan ($GlcNAc_2Man_9$)

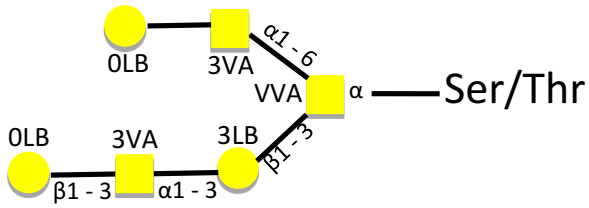


Sequence file for $GlcNAc_2Man_9$:

```
C=P.111/HD22:-(ND2,C1,<Ce>)-4YB-(O4,C1,<a>)-4YB-(O4,C1,<b>)-VMB<branch1>-
(O6,C1,<c>)-VMA<branch2>-(O6,C1,<c>)-2MA-(O2,C1,<Cg>)-0MA

branch1 = C.4:-(O3,C1,<d>)-2MA-(O2,C1,<Cf>)-2MA-(O2,C1,<Cg>)-0MA
branch2 = C.4:-(O3,C1,<d>)-2MA-(O2,C1,<Cg>)-0MA
Ce=ND2,C1,[CB CG ND2 C1 180 CG ND2 C1 C2 130]
a = O4,C1,[C3 C4 O4 C1   60.0 C4 O4 C1 C2 120.0]
b = O4,C1,[C3 C4 O4 C1  100.0 C4 O4 C1 C2  60.0]
c = O6,C1,[C4 C5 C6 O6   120.0 C5 C6 O6 C1 150.0 C6 O6 C1 C2 180.0]
d = O3,C1,[C2 C3 O3 C1  120.0 C3 O3 C1 C2 -60.0]
Cf = O2,C1,[C3 C2 O2 C1 205.0 C2 O2 C1 C2 180.0]
Cg = O2,C1,[C3 C2 O2 C1 -60.0 C2 O2 C1 C2 220.0]
```

B) Mucin Type O-glycan (GalNAc.Gal)$_3$



Sequence file for (GalNAc.Gal)$_3$:

```
RC=P.2018/HG1:-(OG1,C1,<RCe>)-VVA<branchR2>-(O6,C1,<Ra2>)-3VA-
(O3,C1,<Rb2>)-0LB

branchR2 = RC.4:-(O3,C1,<Rc2>)-3LB-(O3,C1,<Rca11>)-3VA-(O3,C1,<Rcb11>)-
0LB
Ra2 = O6,C1,[C5 C6 O6 C1 210.0 C6 O6 C1 C2 60.0]
Rca11 = O3,C1,[C2 C3 O3 C1 125.0 C3 O3 C1 C2 195.0]
Rb2 = O3,C1,[C2 C3 O3 C1  150.0 C3 O3 C1 C2 90.0]
Rcb11 = O3,C1,[C2 C3 O3 C1  150.0 C3 O3 C1 C2 130.0]
Rc2 = O3,C1,[C2 C3 O3 C1  160.0 C3 O3 C1 C2 60.0]
RCe = OG1,C1,[CB OG1 C1 C2  68.0 OG1 C1 C2 C3 70.
```

3. Examples
3.1 Carbohydrate polymers

Here, we show how to prepare and set up MD simulations for carbohydrate polymers using the *doGlycans* tool. The user has to provide only the sequence file for the carbohydrate polymer. The tool then generates files for MD simulations compatible with the OPLS-AA, AMBER, and CHARMM force fields.

To this end one has to generate the sequence file of the polymer one is interested and then simply execute the following command:

```
python $DGPATH/prepreader.py [-Wmdt] -f $DGPATH/dat/glycam06h.itp -p
$DGPATH/dat/GLYCAM_06h.prep -s sequence_file.seq -c POL.pdb -o POL.itp --
(force field)
```

Options:

-h	Show this help message and exit
-f	GLYCAM force field parameter file converted to the GROMACS format (input) [default: [glycam06h.itp]]
-p	GLYCAM residue database file (input) [default: [GLYCAM_06h.prep]]
-s	Sequence file for sugars with specified linkages (input) [default: none]
-m	Generate PDBs and topology for monomers [default: False]
-d	Generate PDBs and topology for dimers [default: False]
-t	Generate PDBs and topology for trimers [default: False]
-W	Suppress warnings [default: False]
-c	Polysaccharide structure file (output) [default: none]
-o	GROMACS topology file (output) [default: none]
--opls	Add suffix '_S' for OPLS atom types [default: True]
--charmm	Print PDB for CHARMM-GUI submission [default: none]
--amber	Remove suffix '_S' for AMBER atom types [default: False]

The above command therefore reads the GLYCAM residue database and builds polysaccharides specified by the sequence file.

3.1.1 Cellulose examples

As an example, we show how to set up cellulose simulations, where all the user has to do is to prepare a sequence file with the required number of D-glucose units and then run the `prepreader.py` script. In practice, the cellulose sequence file for a linear chain of 15 D-glucose (β-1,4 linked) units is written as below and provided as an input file (file name: `sequence_file.seq`) to the `prepreader.py` script:

Sequence file for cellulose:

```
Cellulose = ROH-(O1,C1)-{4GB-(O4,C1,<a>)}14-0GB
a = O4,C1,[H4 C4 O4 C1 0.0 C4 O4 C1 H1 60.0]
```

After the sequence file has been generated, one can continue to build the carbohydrate chain in the following fashion:

A) For the OPLS-AA force field:

```
python $DGPATH/prepreader.py [-Wmdt] -f $DGPATH/dat/glycam06h.itp -p $DGPATH/dat/GLYCAM_06h.prep -s sequence_file.seq -c POL.pdb -o POL.itp --opls
```

The command reads the GLYCAM force field parameter file converted to GROMACS format (`glycam06h.itp`), GLYCAM residue database (`GLYCAM_06h.prep`) and builds polysaccharides specified by the sequence file (`sequence_file.seq`). On using the "`--opls`" flag in the above command, the *doGlycans* tool writes the structure to a coordinate file (`POL.pdb`) and the GROMACS topology to an ITP (include topology) file (`POL.itp`) for the OPLS-AA force field.

B) For the AMBER force field:

```
python $DGPATH/prepreader.py [-Wmdt] -f $DGPATH/dat/glycam06h.itp -p $DGPATH/dat/GLYCAM_06h.prep -s sequence_file.seq -c POL.pdb -o POL.itp --amber
```

The command reads the GLYCAM force field parameter file converted to GROMACS format (`glycam06h.itp`), GLYCAM residue database (`GLYCAM_06h.prep`) and builds polysaccharides specified by the sequence file (`sequence_file.seq`). On using the "`--amber`" flag in the above command, the *doGlycans* tool writes the structure to a coordinate file (`POL.pdb`) and the GROMACS topology to an ITP (include topology) file (`POL.itp`) for the AMBER force field.

C) For the CHARMM force field:

```
python $DGPATH/prepreader.py [-Wmdt] -f $DGPATH/dat/glycam06h.itp -p $DGPATH/dat/GLYCAM_06h.prep -s sequence_file.seq -c POL.pdb -o POL.itp --charmm
```

The command reads the GLYCAM force field parameter file converted to GROMACS format (`glycam06h.itp`), GLYCAM residue database (`GLYCAM_06h.prep`) and builds polysaccharides specified by the sequence file (`sequence_file.seq`). On using the "`--charmm`" flag in the above command, it writes the structure file (`POL_charmm-gui.pdb`), which can be submitted to CHARMM-GUI ("`Quick MD simulator`") to generate CHARMM force field files.

In the *doGlycans* package (`/doglycans/Examples/`), we provide all the files required to prepare and run cellulose simulations using the OPLS-AA, AMBER, and CHARMM force field. Users can execute the commands to generate cellulose structure and their topology file using the cellulose sequence provided in the path: `/doglycans/Examples/CELLULOSE` and then compare the output with the files provided here: `/doglycans/Examples/CELLULOSE/forcefield`.

4. Glycosylation of proteins

Here we show how to glycosylate epidermal growth factor receptor (EGFR) by adding N-linked glycans to the receptor. The description demonstrates glycosylation in three steps, where the first step processes the protein data bank (PDB) file, then the second step prepares the sequence file, and finally the last step shows how to run the script for glycosylation.

The script can also be used to covalently link O-linked glycans. As a test case, we therefore demonstrate as a second example the linking of both N- and O- glycans to the human CD59 glycoprotein.

4.1 Processing the Protein Data Bank file using the GROMACS tool `pdb2gmx`

```
pdb2gmx –f Prot.pdb –o ProtH.pdb –p ProtH.top
```

When prompted, choose the OPLS-AA parameter set for simulations with the OPLS-AA force field, or choose the AMBER parameter set for simulations with the AMBER force field. Next, the tool prompts to choose the water model, where the TIP3P water model is recommended. In the above command, `pdb2gmx` reads the input `Prot.pdb` file and then writes the structure file (`ProtH.pdb`) with all hydrogen atoms included, and the corresponding topology file (`ProtH.top`).

The `ProtH.top` file generated by `pdb2gmx` requires some modification to be read by the *doGlycans* tool. First, move (or rename) the file as follows:

```
mv ProtH.top ProtH.itp
```

Next, in the `ProtH.itp` file comment out the include statement for the force field. Here we have chosen the OPLS-AA parameter set as an example. If you have selected the AMBER parameter set, then you should have force field information related to the AMBER force field. The syntax is as follows:

```
; Include forcefield parameters
; #include "oplsaa.ff/forcefield.itp"
```

Also, comment out or delete all the lines starting from the protein position restraint section at the end of the `ProtH.itp` file:

```
; Include Position restraint file
;#ifdef POSRES
;#include "posre.itp"
;#endif
```

```
; Include water topology
;#include "oplsaa.ff/tip3p.itp"

;#ifdef POSRES_WATER
; Position restraint for each water oxygen
;[ position_restraints ]
;  i funct       fcx        fcy        fcz
;  1    1       1000       1000       1000
;#endif

; Include topology for ions
;#include "oplsaa.ff/ions.itp"

;[ system ]
; Name
;Generated by trjconv : egfr-tether-LdLo-with-sugars t=   0.00000

;[ molecules ]
; Compound        #mols
;Protein             1
```

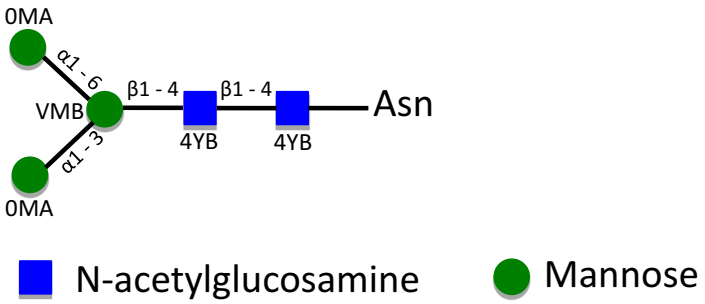
4.2 Preparing the sequence file

Both N- and O-glycan sequences can be submitted in the same sequence file. The sequence file is the final input file required for generating a glycosylated protein structure. An an example, the sequence files for saccharides attached to EGFR and CD59 are shown below.

4.2.1 Preparing the sequence file for glycosylation of EGFR

Below is the example sequence for the attachment of Mannose type N-glycan ($GlcNAc_2Man_3$) to the 10 glycosylation sites of EGFR (residue numbers 32, 151, 172, 328, 337, 389, 420, 504, 544, and 579) by replacing the hydrogen atom HD22 of Asparagine residues. Here the first line for each glycosylation site (uniquely labeled for each residue) consists of information on the saccharide and its linkage. The first linkage is between the ND2 atom of Asparagine and the C1 atom of N-acetylglucosamine (the first sugar). The rest of the nomenclature and linkages follows the GLYCAM naming convention. After each linkage, a unique label within the angle brackets specifies the connections between the two atoms as well as the dihedral angles with their values. The information about the dihedral connections and their values are given on separate lines. Since in EGFR we link the same mannose type N-glycan ($GlcNAc_2Man_3$) to all the 10 glycosylation sites, it is not required to define the dihedral connections and their values for each site separately. The *doGlycans* tool will read the unique label within the angle brackets and assign the dihedral values accordingly at each glycosylation site. However, if the glycan added is complex and differs for each site, then a separate dihedral value can be set for each glycosylation site (see the sequence file for CD59).

Sequence file for glycosylation of EGFR with Mannose type N-glycan ($GlcNAc_2Man_3$):



```
A=P.32/HD22:-(ND2,C1,<Ae>)-4YB-(O4,C1,<a>)-4YB-(O4,C1,<b>)-VMB<branch>-
(O6,C1,<c>)-0MA

branch = A.4:-(O3,C1,<d>)-0MA
a = O4,C1,[C3 C4 O4 C1   60.0 C4 O4 C1 C2 120.0]
b = O4,C1,[C3 C4 O4 C1  100.0 C4 O4 C1 C2  60.0]
c = O6,C1,[C4 C5 C6 O6   120.0 C5 C6 O6 C1 150.0 C6 O6 C1 C2 180.0]
d = O3,C1,[C2 C3 O3 C1  120.0 C3 O3 C1 C2 -60.0]
Ae=ND2,C1,[CG ND2 C1 C2 230]

B=P.151/HD22:-(ND2,C1,<Ae>)-4YB-(O4,C1,<a>)-4YB-(O4,C1,<b>)-VMB<branch>-
(O6,C1,<c>)-0MA

C=P.172/HD22:-(ND2,C1,<Ae>)-4YB-(O4,C1,<a>)-4YB-(O4,C1,<b>)-VMB<branch>-
(O6,C1,<c>)-0MA

D=P.328/HD22:-(ND2,C1,<Ae>)-4YB-(O4,C1,<a>)-4YB-(O4,C1,<b>)-VMB<branch>-
(O6,C1,<c>)-0MA

E=P.337/HD22:-(ND2,C1,<Ae>)-4YB-(O4,C1,<a>)-4YB-(O4,C1,<b>)-VMB<branch>-
(O6,C1,<c>)-0MA

F=P.389/HD22:-(ND2,C1,<Ae>)-4YB-(O4,C1,<a>)-4YB-(O4,C1,<b>)-VMB<branch>-
(O6,C1,<c>)-0MA

G=P.420/HD22:-(ND2,C1,<Ae>)-4YB-(O4,C1,<a>)-4YB-(O4,C1,<b>)-VMB<branch>-
(O6,C1,<c>)-0MA

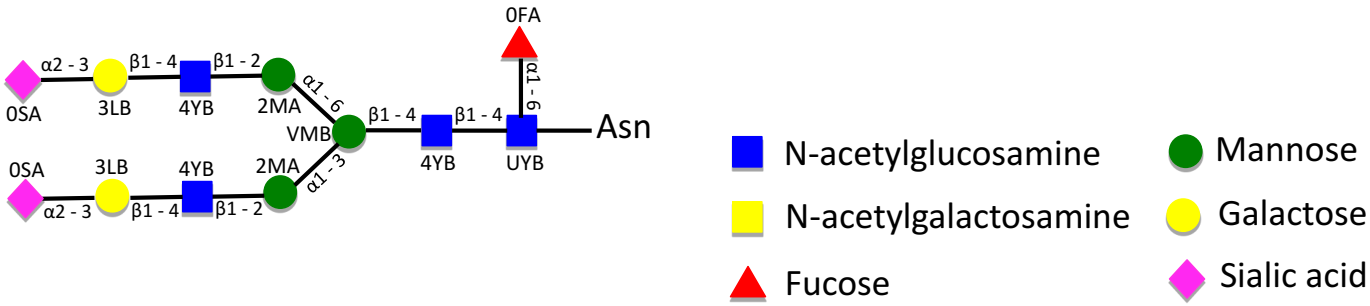
H=P.504/HD22:-(ND2,C1,<Ae>)-4YB-(O4,C1,<a>)-4YB-(O4,C1,<b>)-VMB<branch>-
(O6,C1,<c>)-0MA

I=P.544/HD22:-(ND2,C1,<Ae>)-4YB-(O4,C1,<a>)-4YB-(O4,C1,<b>)-VMB<branch>-
(O6,C1,<c>)-0MA

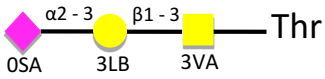
J=P.579/HD22:-(ND2,C1,<Ae>)-4YB-(O4,C1,<a>)-4YB-(O4,C1,<b>)-VMB<branch>-
(O6,C1,<c>)-0MA
```

4.2.2 Preparing the sequence file for N- and O- linked glycosylation of CD59

a) Complex Type N-glycan ($GlcNAc_4Man_3Gal_2FucNeuNAc_2$):



b) Mucin Type O-glycan (GalNAcGalNeuNAc):



Sequence file for glycosylation of CD59 with complex type N-glycan ($GlcNAc_4Man_3Gal_2FucNeuNAc_2$) linked at residue 18 and the Mucin type O-glycan (GalNAcGalNeuNAc) linked at residue 52:

```
E=P.18/HD22:-(ND2,C1,<Eh>)-UYB<EbranchFuc>-(O4,C1,<Ea>)-4YB-(O4,C1,<Eb>)-
VMB<EbranchMan>-(O6,C1,<Ec>)-2MA-(O2,C1,<Ed>)-4YB-(O4,C1,<Ei>)-3LB-
(O3,C2,<Ej>)-0SA

EbranchFuc = E.4:-(O6,C1,<Ee>)-0FA
EbranchMan = E.4:-(O3,C1,<Ef>)-2MA-(O2,C1,<Eg>)-4YB-(O4,C1,<Ek>)-3LB-
(O3,C2,<El>)-0SA
Ea = O4,C1,[C3 C4 O4 C1   119.364 C4 O4 C1 C2 159.284]
Eb = O4,C1,[C3 C4 O4 C1  109.335 C4 O4 C1 C2  156.828]
Ec = O6,C1,[C4 C5 C6 O6   63.058 C5 C6 O6 C1 142.635 C6 O6 C1 C2 -
177.591]
Ed = O2,C1,[C3 C2 O2 C1  119.036 C2 O2 C1 C2  148.221]
Ee = O6,C1,[C4 C5 C6 O6   68.073 C5 C6 O6 C1 -143.104 C6 O6 C1 C2
142.538]
Ef = O3,C1,[C2 C3 O3 C1  -87.136 C3 O3 C1 C2 -167.302]
Eg = O2,C1,[C3 C2 O2 C1  176.592 C2 O2 C1 C2  -173.747]
Ei = O4,C1,[C3 C4 O4 C1  109.335 C4 O4 C1 C2  156.828]
Ej = O3,C2,[C2 C3 O3 C2  -87.136 C3 O3 C2 C3 -167.302]
Ek = O4,C1,[C3 C4 O4 C1  109.335 C4 O4 C1 C2  156.828]
El = O3,C2,[C2 C3 O3 C2  -87.136 C3 O3 C2 C3 -167.302]
Eh=ND2,C1,[CB CG ND2 C1 78.223 CG ND2 C1 C2 61.491]


RD=P.52/HG1:-(OG1,C1,<RDe>)-3VA-(O3,C1,<Rb3>)-3LB-(O3,C2,<Rbd>)-0SA

Rb3 = O3,C1,[C2 C3 O3 C1  145.0 C3 O3 C1 C2 100.0]
Rbd = O3,C2,[C2 C3 O3 C2  -87.136 C3 O3 C2 C3 -167.302]
RDe=OG1,C1,[CB OG1 C1 C2 60 OG1 C1 C2 C3 115 CG2 CB OG1 C1 30]
```

4.3 Adding sugars to the structure file using the *doGlycans* tool

Below is the command to generate a glycosylated protein. The command `doglycans.py -h` lists the options for input and output files. The input file `glycam06h.itp` includes `Glycam_06_H` parameters converted to the GROMACS format, and `GLYCAM_06h.prep` includes sugar residues. The input files `ProtH.pdb` and `ProtH.itp` were prepared in step 1 and the sequence file in step 2. The user can choose the force field available from the list using the flag "`--opls`" or "`--amber`" or "`--charmm $DGPATH/dat/GLYCAM_06h_resname.txt`".

Command:

```
python $DGPATH/doglycans.py -f $DGPATH/dat/glycam06h.itp -p
$DGPATH/dat/GLYCAM_06h.prep -s sequence_file.seq -c ProtH.pdb -i
ProtH.itp -o ProtG.pdb -t ProtG.itp -B --(force field)
```

Options:

-h	Show this help message and exit
-f	GLYCAM force field parameter file converted to the GROMACS format (input) [default: [glycam06h.itp]]
-p	GLYCAM residue database file (input) [default: [GLYCAM_06h.prep]]
-s	Sequence file for sugars with specified linkages (input) [default: none]
-c	Protein structure file in the PDB format [default: none]
-o	Glycosylated structure file (output) [default: none]
-W	Suppress warnings [default: False]
-i	GROMACS topology file (input) [default: none]
-t	GROMACS topology file (output) [default: none]
-g	Add prefix 'G_' for atom types [default: False]
-B	Do not perceive bonds from atom distances [default: False]
--opls	Add suffix '_S' for OPLS atom types [default: True]
--charmm=PATHFN	Print PDB for CHARMM-GUI submission. The path to a file with a sugar residue name list is needed [default: none]
--amber	Remove suffix '_S' for AMBER atom types [default: False]

After the sequence file has been generated, one can continue to build the glycosylated protein in the following fashion:

A) For the OPLS-AA force field:

```
python $DGPATH/doglycans.py -f $DGPATH/dat/glycam06h.itp -p
$DGPATH/dat/GLYCAM_06h.prep -s sequence_file.seq -c ProtH.pdb -i
ProtH.itp -o ProtG.pdb -t ProtG.itp -B --opls
```

The command reads the GLYCAM force field parameter file converted to GROMACS format (`glycam06h.itp`), GLYCAM residue database (`GLYCAM_06h.prep`), protein coordinate file (`ProtH.pdb`), and topology file (`ProtH.itp`), and builds glycosylated protein structure using the

specified sequence file (`sequence_file.seq`). As an output, the *doGlycans* tool returns the glycosylated structure file in the PDB format (`ProtG.pdb`) and the topology file (`ProtG.itp`) in the GROMACS .itp format for the OPLS-AA force field.

The output PDB file has to be checked for any steric clashes between the sugar and the protein residues. If there are minor steric clashes, then the energy minimization of the system will remove any steric hindrance. However, if there are severe steric clashes, then the dihedral angles have to be adjusted in the sequence file and one has to rerun the `doglycans.py` tool to generate the structure file with no or reduced atomic clashes before running the energy minimization procedure again. Next, the generated GROMACS topology file has to be edited such that the atom types and charges are adjusted for those protein atoms that are involved in the covalent linkage to the glycans (see Section 4.4).

In the *doGlycans* package (`/doglycans/Examples/`), we provide all the files required to prepare and run EGFR and CD59 simulations using the OPLS-AA force field. Users can execute the commands to generate EGFR/CD59 structure (`ProtG.pdb`) and their topology file (`ProtG.itp`) using the input sequence, protein coordinate file (`ProtH.pdb`), and topology file (`ProtH.itp`) provided in their paths: `/doglycans/Examples/EGFR(CD59)/OPLS_AAff`. We also provide the output structure file (`ProtG.pdb`) and the modified topology file (`ProtG.itp`), which the users can use as reference while editing the topology file. To avoid overwriting of output files, users can run the `doglycans.py` script with the input files from a different directory.

B) For the AMBER force field:

```
python $DGPATH/doglycans.py -f $DGPATH/dat/glycam06h.itp -p
$DGPATH/dat/GLYCAM_06h.prep -s sequence_file.seq -c ProtH.pdb -i
ProtH.itp -o ProtG.pdb -t ProtG.itp -B --amber
```

The command reads the GLYCAM force field parameter file converted to the GROMACS format (`glycam06h.itp`), GLYCAM residue database (`GLYCAM_06h.prep`), protein coordinate file (`ProtH.pdb`), and topology file (`ProtH.itp`), and builds glycosylated protein structure using the specified sequence file (`sequence_file.seq`). As an output, the *doGlycans* tool returns the glycosylated structure file in the PDB format (`ProtG.pdb`) and the topology file (`ProtG.itp`) in the GROMACS .itp format for the AMBER force field.

The output PDB file has to be checked for any steric clashes between the sugar and protein residues. If there are minor steric clashes, then the energy minimization of the system will remove any steric hindrance. However, if there are severe steric clashes, then the dihedral angles have to be adjusted in the sequence file and one has to rerun the `doglycans.py` tool to generate the structure file with no or reduced atomic clashes before running the energy minimization procedure again. Next, the generated GROMACS topology file has to be edited such that the atom types and charges are adjusted for those protein atoms that are involved in the covalent linkage to the glycans (see Section 4.5).

In the *doGlycans* package (`/doglycans/Examples/`), we provide all the files required to prepare and run EGFR and CD59 simulations using the AMBER force field. Users can execute the commands to generate EGFR/CD59 structure (`ProtG.pdb`) and their topology file (`ProtG.itp`) using the input sequence, protein coordinate file (`ProtH.pdb`), and topology file (`ProtH.itp`)

provided in their paths: `/doglycans/Examples/EGFR(CD59)/AMBERff`. We also provide the output structure file (`ProtG.pdb`) and the modified topology file (`ProtG.itp`), which the users can use as reference while editing the topology file. To avoid overwriting of output files, users can run the `doglycans.py` script with the input files from a different directory.

C) For the CHARMM force field:

```
python $DGPATH/doglycans.py -f $DGPATH/dat/glycam06h.itp -p
$DGPATH/dat/GLYCAM_06h.prep -s sequence_file.seq -c ProtH.pdb -i
ProtH.itp -o ProtG.pdb -t ProtG.itp -B --charmm
$DGPATH/dat/GLYCAM_06h_resname.txt
```

The command reads the GLYCAM force field parameter file converted to the GROMACS format (`glycam06h.itp`), GLYCAM residue database (`GLYCAM_06h.prep`), protein coordinate file (`ProtH.pdb`), topology file (`ProtH.itp`), and sugar residue list (`GLYCAM_06h_resname.txt`), and builds the glycosylated structure file in the PDB format (`ProtG_charmm-gui.pdb`) using the specified sequence file (`sequence_file.seq`). The generated `ProtG_charmm-gui.pdb` can be submitted to CHARMM-GUI to generate CHARMM force field files.

Note: Users are requested to first visualize the structure file for any steric clashes. If there are no steric clashes, then the *doGlycans* generated structure can be directly submitted to CHARMM-GUI. If there are steric clashes, then the users are requested to minimize the structure file to remove the clashes before submitting to CHARMM-GUI again.

Below are the steps to minimize the *doGlycans* generated structure using GROMACS tools and then prepare the structure file for CHARMM-GUI.

C.1) Energy-minimization to remove any steric clashes:

```
grompp -f minimization.mdp -c ProtG_charmm-gui.pdb -p ./top/system.top -
o ProtG_charmm-gui_minim.tpr

mdrun -v -deffnm ProtG_charmm-gui_minim
```

C.2) If there is no chain ID in the structure file generated from the above step A, then it can be added as follows:

```
editconf -f ProtG_charmm-gui_minim.gro -o ProtG_charmm-gui_minim.pdb -
label A
```

C.3) In the *doGlycans* package, we also provide the `doglycans2charmmGui.py` script to generate a structure file compatible with CHARMM-GUI. Here the user has to make sure that the chain ID is included to all the sugar residues in the structure file and then use the `doglycans2charmmGui.py` script.

```
python $DGPATH/doglycans2charmmGui.py -c ProtG_charmm-gui_minim.pdb -i
ProtG.itp --charmm $DGPATH/dat/GLYCAM_06h_resname.txt
```

Here, the `doglycans2charmmGui.py` script reads the user-provided minimized glycosylated structure (`ProtG_charmm-gui_minim.pdb`) and the glycosylated topology file (`ProtG.itp`) files and writes a new structure file (`ProtG_charmm-gui_minim_charmm-gui.pdb`), which can be read by CHARMM-GUI. Importantly, when processing the structure file in CHARMM-GUI, users are requested to check if the sugars, linkages, and configurations generated by CHARMM-GUI match the sequence file. If not, then CHARMM-GUI provides the option of editing the glycan sequence.

In the *doGlycans* package (`/doglycans/Examples/`), we provide all the files required to prepare and run EGFR and CD59 simulations using the CHARMM force field. Users can execute the commands to generate EGFR/CD59 structure (`ProtG_charmm-gui.pdb`) using the input sequence, protein coordinate file (`ProtH.pdb`), and topology file (`ProtH.itp`) provided in their paths: `/doglycans/Examples/EGFR(CD59)/CHARMMff`. We also provide the output structure file (`ProtG_charmm-gui.pdb`) which the users can use as reference while preparing input structure file for CHARMM-GUI. To avoid overwriting of output files, users can run the `doglycans.py` script with the input files from a different directory.

4.4 Preparing the topology file for MD Simulations: the OPLS-AA force field

The GROMACS topology file generated in Section 4.3 requires some manual modifications, such as:

- Changing atom types in the protein for attachment points
- Changing charges
- Adding explicit bonded parameters for cross-links

For N-linked glycans:

- Changing atom types in the protein for the attachment points of N and H
- Changing charges of N and H (and adjusting those to get integer charges for the protein)

Example: If a sugar is linked to the Asn residue at the position 16, then the OPLS atom types and charges for the atoms ND2 and HD21 are changed from:

```
238    opls_237     16    ASN     ND2     83      -0.76      14.0067  ; qtot
239    opls_240     16    ASN    HD21     83       0.38       1.008   ; qtot
```

to read:

```
238    opls_238     16    ASN     ND2     83      -0.50      14.0067  ; qtot
239    opls_241     16    ASN    HD21     83       0.30       1.008   ; qtot
```

For O-linked glycans:

- Changing atom types in the protein for the attachment points of OG/OG1 for Ser/Thr, respectively
- Changing charges of OG/OG1 (and adjusting those to get integer charges for the protein)

Example: If a sugar is linked to the Thr residue at the position 9, then the OPLS atom types and charges for the atom OG1 are changed from:

```
110     opls_154       9     THR     OG1      36     -0.6830      15.999 ; qtot
```

to read:

```
110    opls_186       9     THR     OG1      36     -0.4000      15.999 ; qtot
```

A similar change is also done for the Ser-linkage. For more information on the OPLS atom types and charges, the users are requested to look into the `atomtypes.atp` and `ffnonbonded.itp` files in the `oplsaa.ff` directory.

The final step in the modification is to add explicit bonded parameters for cross-links, however users are not required to do anything here as the cross-link parameters for both N- and O- linked glycans (`types_N_linked_glycan.itp`, `types_O_linked_glycan.itp`) are provided in the package. Moreover, also the non-bonded parameters (`nb_params.itp`) and pair types (`pairtypes.itp`) are included in the package.

Importantly, users are required to check the total charge of the protein after charge modifications. The total charge of the protein before and after glycosylation (except when charged saccharide units such as sialic acid are present) should be the same. In the case of total charge variation, the charge has to be equally spread across all the linked residues and sugars. However, if the negatively charged sialic acid (Neu5Ac) were linked to the protein, then the system would carry an additional charge of -1e for each sialic acid unit. In such cases, the charge from the sialic acid units should be neutralized using ions.

4.5 Preparing the topology file for MD Simulations: the AMBER force field

The GROMACS topology file generated in Section 4.3 requires some manual modifications, such as:

- Changing atom types in the protein for attachment points
- Changing charges

For N-linked glycans:

- Changing charge in the protein for the attachment point of ND2

Example: If a sugar is linked to the Asn residue at the position 18, then the charges for the atom ND2 is changed from:

```
238          N      18     ASN     ND2     238     -0.9191      14.007 ; qtot
```

to read:

```
238          N      18     ASN     ND2     238     -0.6935      14.007 ; qtot
```

For O-linked glycans:

- Changing atom types and charges in the protein for the attachment points of OG/OG1 for Ser/Thr, respectively

Example: If a sugar is linked to the Thr residue at the position 52, then the atom type and charges for the atom OG1 are changed from:

```
25915         OH    52     THR     OG1   25951     -0.6761      15.999 ; qtot
```

to read:

```
25915        Os     52     THR     OG1   25951     -0.4599      15.999 ; qtot
```

A similar change is also done for the Ser-linkage. For more information on the atom types and charges, the users are requested to look into the `GLYCAM_06h.prep` file which contains parameters for Asparagine when N-linked to sugars, and Serine/Threonine when O-linked to sugars.

After adjusting the atom types and charges, the system can be subjected to energy minimization and subsequently to equilibration and production runs using GROMACS simulations package. No additional files are required.

When running the GROMPP step for energy minimization, a warning message is shown as below:

```
WARNING 1 [file glycam06h.itp, line 150]:
  Overriding Angle parameters.

  old: 102.6 376.56 102.6 376.56
  new: Os     P    Os  1   109.600  376.560
```

The warning message can be ignored using the "`-maxwarn 1`" flag as the GLYCAM value is recommended.

Similar to OPLS-AA simulations, users are required here to check the total charge of the protein after charge modifications. The total charge of the protein before and after glycosylation (except when charged saccharide units such as sialic acid are present) should be the same. In the case of total charge variation, the charge has to be equally spread across all the linked residues and sugars. However, if the negatively charged sialic acid (Neu5Ac) were linked to the protein, then the system

would carry an additional charge of -1e for each sialic acid unit. In such cases, the charge from the sialic acid units should be neutralized using ions.

5. Procedures to prepare glycans for glycolipids

To prepare glycolipid structures for MD simulations, the user has to provide the ceramide structure and its topology file. The preparation of the sequence file for glycolipids follows essentially the same approach as in Section 2 (sequence preparation). The sequence file can be easily modified to generate GM1, GM2, GM3, or any other sugar head groups. The user can choose between the force fields of choice using the flag "`--opls`", "`--amber`" or "`--charmm ./doglycans/dat/GLYCAM_06h_resname.txt`". The *doGlycans* tool reads the input files and generates glycolipid structure and topology files. The main additional feature that one should pay attention to regards the atom types and charges, which need to be adjusted according to the force field used for lipids. In the below command, we show how the `doglycans.py` tool can be used to covalently link the oligosaccharide domain (Gal5-β1,3-GalNAc4-β1,4-(NeuAc3-α2,3)-Gal2-β1,4-Glc1-β1) to the ceramide domain to generate GM1.

An example sequence file for an oligosaccharide domain attached to ceramide is shown below.

```
N=P.1/H11:-(O1,C1,<a>)-4GB-(O4,C1,<b>)-WLB<branch>-(O4,C1,<d>)-3VB-
(O3,C1)-0LB

branch =N.4:-(O3,C2,<c>)-0SA
a=O1,C1,[C3 C2 O1 C1 180.0]
b=O4,C1,[C2 C3 C4 O4 175 C3 C4 O4 C1 112 C4 O4 C1 C2 156 O4 C1 C2 C3 170]
c=O3,C2,[C2 C3 O3 C2 -90.0]
d=O4,C1,[C4 O4 C1 O5 -120.0]
```

After the sequence file has been generated, one can continue to build the glycolipid in the following fashion.

Command:

```
python $DGPATH/doglycans.py -f $DGPATH/dat/glycam06h.itp -p $DGPATH
/dat/GLYCAM_06h.prep -s sequence_file.seq -c CER.pdb -i CER.itp -o
GLIP.pdb -t GLIP.itp -B --(force field)
```

Options:

`-h`	Show this help message and exit
`-f`	GLYCAM force field parameter file converted to the GROMACS format (input) [default: [`glycam06h.itp`]]
`-p`	GLYCAM residue database file (input) [default: [`GLYCAM_06h.prep`]]
`-s`	Sequence file for sugars with specified linkages (input) [default: none]

-c	Protein structure file in the PDB format [default: none]
-o	Glycosylated structure file (output) [default: none]
-W	Suppress warnings [default: False]
-i	GROMACS topology file (input) [default: none]
-t	GROMACS topology file (output) [default: none]
-g	Add prefix 'G_' for atom types [default: False]
-B	Do not perceive bonds from atom distances [default: False]
--opls	Add suffix '_S' for OPLS atom types [default: True]
--charmm=PATHFN	Print PDB for CHARMM-GUI submission. The path to a file with a sugar residue name list is needed [default: none]
--amber	Remove suffix '_S' for AMBER atom types [default: False]

A) For the OPLS-AA force field:

```
python $DGPATH/doglycans.py -f $DGPATH/dat/glycam06h.itp -p
$DGPATH/dat/GLYCAM_06h.prep -s sequence_file.seq -c CER.pdb -i CER.itp -o
GLIP.pdb -t GLIP.itp -B --opls
```

The command reads the GLYCAM force field parameter file converted to the GROMACS format (`glycam06h.itp`), GLYCAM residue database (`GLYCAM_06h.prep`), ceramide coordinate file (`CER.pdb`), and ceramide topology file (`CER.itp`), and builds GM1 structure using the specified sequence file (`sequence_file.seq`). As an output, the *doGlycans* tool returns the GM1 structure file in the PDB format (`GLIP.pdb`) and the topology file (`GLIP.itp`) in the GROMACS .itp format for the OPLS-AA force field. The generated `GLIP.itp` file is further modified such that the GLYCAM naming for sugars is replaced with OPLS atom types, and charges are adjusted appropriately.

In the *doGlycans* package (`/doglycans/Examples/`), we provide all the files required to prepare and run GM1 simulations using the OPLS-AA force field. Users can execute the commands to generate GM1 structure and their topology file using the input sequence file, ceramide coordinate file (`CER.pdb`), and topology file (`CER.itp`) provided in the path: `/doglycans/Examples/GM1`. We also provide the output structure file (GLIP.pdb) and the modified topology file (`GLIP.itp`) for the OPLS-AA force field (`/doglycans/Examples/GM1/OPLS_AAff`), which the users can use as reference while editing the topology file. To avoid overwriting of output files, users can run the `doglycans.py` script with the input files from a different directory.

B) For the AMBER force field:

For the AMBER force field, user has to provide an AMBER compatible ceramide structure and topology files as input and the *doGlycans* tool will then generate glycolipid structures for simulations carried out with the AMBER force field.

```
python $DGPATH/doglycans.py -f $DGPATH/dat/glycam06h.itp -p
$DGPATH/dat/GLYCAM_06h.prep -s sequence_file.seq -c CER.pdb -i CER.itp -o
GLIP.pdb -t GLIP.itp -B --amber
```

C) For the CHARMM force field:

For the CHARMM force field, users should follow the same steps as described in Section 4.3.C on how to generate the structure file for CHARMM-GUI.

```
python $DGPATH/doglycans.py -f $DGPATH/dat/glycam06h.itp -p $DGPATH/dat/GLYCAM_06h.prep -s sequence_file.seq -c CER.pdb -i CER.itp -o GLIP.pdb -t GLIP.itp -B --charmm $DGPATH/dat/GLYCAM_06h_resname.txt
```

The above command generates a structure file "`GLIP_charmm-gui.pdb`".On submitting the *doGlycans* generated glycolipid structure file (`GLIP_charmm-gui.pdb`) to CHARMM-GUI, the server reads the sugars as a glycan type and ceramide as a hetero type. Ceramide can be ignored while processing the input structure on the server, since in the follow-up steps CHARMM-GUI provides the option of editing the glycan sequence and choosing the available ceramides for the CHARMM force field.

Moreover, the structure file for (only) the GM1 sugar head group can be generated using `prepreader.py` and submitted to CHARMM-GUI to generate glycolipids for the CHARMM force field. Below is an example sequence file to generate only the oligosaccharide domain and the command to write the structure file (`GLIP_charmm-gui.pdb`) for CHARMM-GUI:

Sequence:

```
GM1=ROH-(O1,C1)-4GB-(O4,C1,<b>)-WLB<branch>-(O4,C1,<d>)-3VB-(O3,C1)-0LB
branch =N.4:-(O3,C2,<c>)-0SA

b=O4,C1,[C2 C3 C4 O4 175 C3 C4 O4 C1 112 C4 O4 C1 C2 156 O4 C1 C2 C3 170]
c=O3,C2,[C2 C3 O3 C2 -90.0]
d=O4,C1,[C4 O4 C1 O5 -120.0]
```

Command:

```
python $DGPATH/prepreader.py [-Wmdt] -f $DGPATH/dat/glycam06h.itp -p $DGPATH/dat/GLYCAM_06h.prep -s GM1_head.seq -c GLIP.pdb -o GLIP.itp --charmm
```

In the *doGlycans* package (`/doglycans/Examples/`), we provide all the files required to prepare and run GM1 simulations using the CHARMM force field. Users can execute the commands to generate GM1 structure using the input sequence file provided in the path `/doglycans/Examples/GM1` and then compare the output (`GLIP_charmm-gui.pdb`) structure file format provided here: `/doglycans/Examples/GM1/CHARMMff`. To avoid overwriting of output files, users can run the `doglycans.py` script with the input files from a different directory.